Group 6/PackMates

Asher Drake

Darrien Schleifer

Ryan Brander

Ivanna Hidalgo

Alex Stoliarchuk

DIG 4172C

March 12, 2026

Group Project - Back-end Development Phase (Milestone 6)

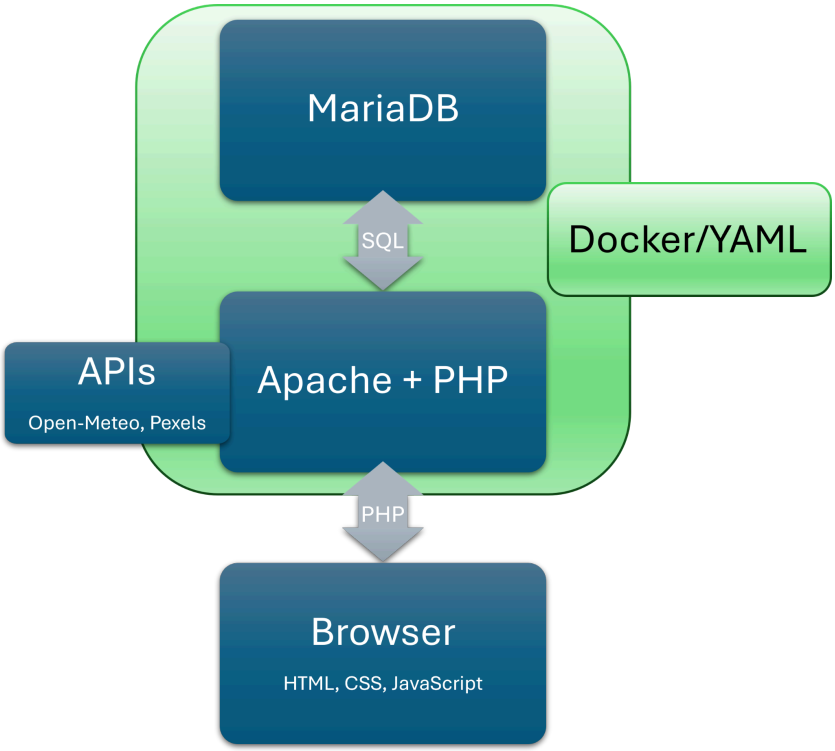
Executive summary:

For this milestone, we fully implemented the back-end for Packmates. The project is feature complete, with a login system, trip creation, packing lists created based on forecasts, and list sharing with invites. We built a MariaDB database that connects to the pages and dynamically sends data back and forth using PHP and SQL queries. We also implemented the usage of the Open-Meteo and Pexels APIs for use in trip creation.

Repository: https://github.com/solarpanel21/packmates.git

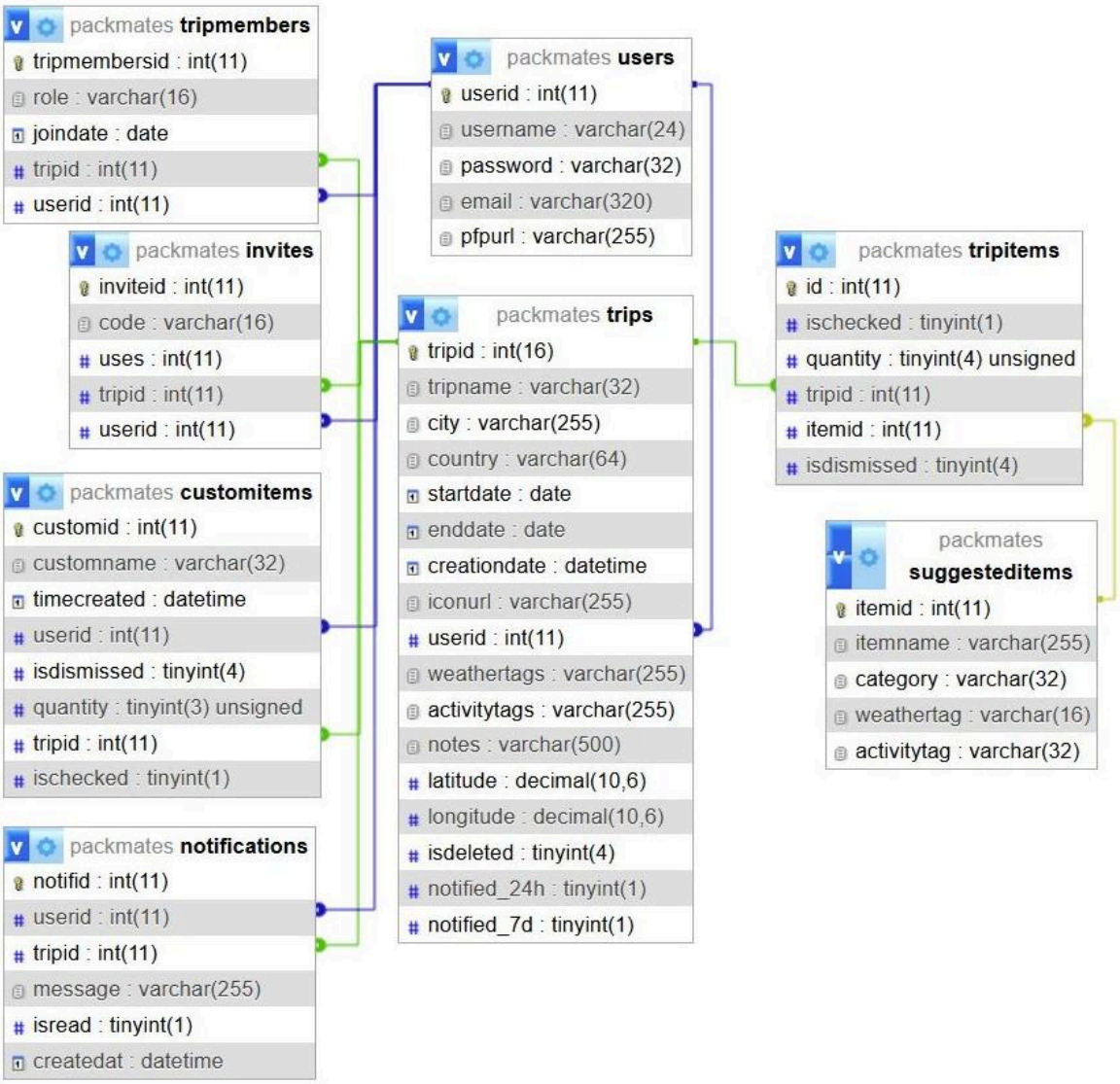
Server-side Architecture Diagram:

Languages used include HTML, CSS, JavaScript, PHP, and SQL. No frameworks or code libraries were used.



Database integration:

Our database management system is MariaDB 10.4, and phpMyAdmin was used to build tables and do most database changes. The data is stored using a relational model, with most tables having foreign keys connecting back to the users table and/or the trips table. The pages connect to the database through a PHP mysqli function in connectionInclude.php, and data is pulled from or inserted into the database depending on user actions and what information a page needs. Deleting information from a user's view is handled by having a column on each piece of information that a user can delete which dictates if it should be hidden from their view (ex: isdeleted column on trips table). This was done rather than allow users to actually remove data from the database for the sake of simplicity.



API usage or development:

For this milestone, we used two main API's, Open-Meteo and Pexels. Open-Meteo's API is split into several different parts, and we used the Open-Meteo weather API, the Open-Meteo historical weather API, and the Open-Meteo geocoding API. Google Maps Embed API was also used as an iframe component.

Open-Meteo:

Open-Meteo's API's (https://open-meteo.com/) are used primarily for the process of creating a new trip. Open-Meteo was chosen because it is free, requires no API key, and has all needed information for the core loop, as well as having usage limits well within our requirements.

Open-Meteo Geocoding API (https://geocoding-api.open-meteo.com/v1/search) was used for both the suggested locations in the search bar and for the logic of the core loop to obtain place names based on beginnings of words in the search bar, and then return JSON for the selected place which can then be parsed into variables and inserted into the database and other functions.

Open-Meteo Weather API (https://api.open-meteo.com/v1/forecast) was used to take the variables for latitude and longitude created by parsing the data from the geocoding API as well as the user's submitted date range and return JSON that contains weather forecast information. After being parsed, this data is passed through a function that compares various factors like temperature and estimated precipitation, compares them to set values, and if they reach certain requirements, returns associated tags like "cold" or "rain" that then get inserted into the trips table in the database as a list of strings.

Open-Meteo Historical API (https://archive-api.open-meteo.com/v1/archive) was used for the same purpose as the weather API. The user's selected dates were passed through a function that would check if they are within the weather API's forecast range, and if not they are passed to the historical API, with the only difference being that the historical API was set to return several years of historical data for the same range of dates that is then averaged to get forecast results to turn into tags.

Google Maps Embed:

Google Maps Embed API (https://maps.google.com/maps) was used to have a responsive Google Maps embed on the new trip page. It's only accessed through an iframe embed which receives the latitude and longitude values set as variables based on whatever city the user currently has selected from the search bar. This just provides visual feedback to the user and lets them see what city they have selected in another way. The output is purely visual.

Pexels:

We used the Pexels API from Pexels (https://www.pexels.com/) to display photos of destinations that users select when selecting a travelling destination. The API allows Packmates website to search for images by sending a request to the Pexels photo database. When a user selects a location, such as Miami or Amsterdam, the app sends a request to the photo search endpoint of the Pexels API. This endpoint is designed to find photos based on a keyword, which in this case is the name of the city.

The main endpoint used in the project is the photo search endpoint: https://api.pexels.com/v1/search. The request includes my API key for authorization so that the Pexels system knows the request is coming from my application. After the request is sent, the API returns information about photos that match the search. The data is returned in JSON format, which is a common format used for exchanging data between applications.

The API sends back information about the photos in JSON format. This information includes things like the photographer's name and different sizes of the image. In Packmates, the app takes one of the photos from the response and shows it on the page. Another important detail is that the app only needed one photo from the results returned by the API. Even though the API can return many images, Packmates selects just one photo and displays it on the page. It was done to keep city images simple and focused while still showing a relevant image for the selected destination.

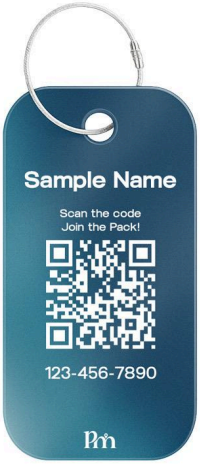
Physical Artifact Integration Report:

The physical artifact developed for this project consists of a luggage tag. This item would contain a QR code that links directly to the Packmates website. The goal and design in mind of this item, is to create a direct link between traveling and the digital platform we developed. By scanning this luggage tag, the user is taken directly to the website and can then manage trips, create packing lists, etc.

From a technical perspective, the QR code on the luggage tag is encoded with the URL of Packmates hosted on our server. When it's scanned, the user's device is sending a HTTP request which loads the Packmates interface through the existing PHP web-based architecture. As for the backend, it then handles authentication, trip retrieval, and packing list data using the MariaDB Database. When the user is logged in, their existing trips and lists are then loaded dynamically through PHP queries and displayed on their dashboard/home page. When the user is not logged in, they are directly sent to the log-in page where they can authenticate their profile to access the features.

This artifact is designed to act as an entry point to the application's core workflow. Once the QR code directs the user to the site, all backend should function normally. This includes trip creation, weather-based packing suggestions (using API's and a list through database queries). This design allows for the physical artifact to connect with real life traveling situations. It provides ease and quick access into the world of Packmates for a much easier and smooth packing experience. Overall, this reinforces the original mission of our application: to help travelers organize and manage packing lists and trips efficiently.

Physical Artifact Mockup:



Testing (and debugging) report:

Bug testing was performed after adding each major feature, making sure it and any other features it closely interacts with were working correctly, and finding the problem if they weren't. Some

major bugs were documented, but there are still some known bugs that will be fixed next milestone (Pexels API not always returning an accurate image, some faulty validation, New Trip page discarding form data, etc.). Although some features were cut, the core loop laid out in previous milestones of creating trips, items, and sharing lists are all present and there should no longer be any fatal bugs that prevent the use of that core loop.

Documented bugs:

Bug	Solution
Duplicate fetch_weather() call in newTrip.php	Removed the second identical fetch_weather() call that was running the weather API twice on every trip creation
file_get_contents disabled in Docker PHP	Added allow_url_fopen = On to the Dockerfile so PHP can make external HTTP requests inside the container
Database hostname hardcoded to localhost	Changed connectionInclude.php to read DB_HOST from an environment variable with a localhost fallback, set DB_HOST=db in compose.yaml for Docker
Activity chips not toggling	Event parameter e was missing from the chip click arrow function causing an undefined variable error, added (e) to the function signature
newTrip page redirecting to home on every load	checkTripNotifications() was placed outside the POST block so it ran on every page load and crashed silently, moved it inside the POST success block

Error message showing on fresh page load	Error display was not wrapped in a POST check so it triggered on GET requests, wrapped in if REQUEST_METHOD === POST check
Shared trips not showing on home page for non-owner users	Changed home page query to also pull trips where the user appears in the tripmembers table
Invite link returned 404 in Docker	Removed hardcoded /packmates/ subfolder from invite URL generation — Docker serves from root, not a subfolder
Destination search always failed in Docker	Changed geocode fetch URL from hardcoded localhost/packmates/geocode.php to a relative path so it works on any server
Countries appearing in destination search results	Added feature_code filter to geocode.php to only return populated place results (PPL) and exclude country-level entries
Spamming save on newTrip.php created multiple duplicate trips	Merged duplicate form submit listeners into one and added a tripSubmitting flag with e.preventDefault() to block repeat submissions
Create Trip button broken after newTrip.php loaded	Old localStorage-based saveTripBtn listener from pre-PHP version was still running and redirecting to tripPreview.html, removed it

Trip creation silently failed with empty fields	Added server-side validation with if/else block to stop execution and show error message when required fields are missing
Custom items not visible to other trip members	Removed userid filter from custom items query so all members of a trip see all custom items, not just their own
Map not updating when destination selected	Fixed selectDestination() JS function which was using undefined bare variables name and country instead of place.name and place.country
Error page shown on newTrip load after PHP error	Moved checkTripNotifications() call inside the POST else block so it only runs after a successful trip insert, not on every page load
Notification SQL error on trip names with apostrophes	Applied real_escape_string() to the full assembled message string instead of just the trip name
tripmembers UNIQUE KEY on userid prevented joining multiple trips	Dropped the unique constraint on userid in the tripmembers table so a user can be a member of more than one trip
Weather API called with 0,0 coordinates when no destination selected	Added separate lat/lon validation check with its own error message distinct from the empty fields check